Advanced Quantitative Research Methods

Time Series and Forecasting

Advanced module A11 — optional self-study

Prof. Dr. Christoph Weisser

Slides, lab notebooks and data:
github.com/ChrisW09/Advanced-Quantitative-Research-Methods

Where this module fits

Course chapter / module	What this module builds on it
Ch. 3 — linear regression	AR models *are* least squares on lagged copies
Ch. 4 — classification	`Weekly`/`Smarket`: time-ordered data we treated as rows
Ch. 5 — resampling	The one pitfall Ch. 5 names but never resolves: CV on time
Ch. 7 — beyond linearity	Seasonal profiles are Chapter 7 curves in disguise

The row that knows its neighbours

Every taught chapter shuffles rows freely, because rows are independent. A time series is the opposite: **the order is the information**. This module makes the order visible (autocorrelation), models it (AR), and evaluates honestly (backtesting).

One of twelve optional advanced modules; self-study. Every number on these slides is reproduced by `make_figures.py` in the module folder.

Contents

1. The Problem
2. Seeing Structure
3. AR Models
4. Honest Evaluation
5. A Financial Postscript
6. Python Lab
7. Summary

Optional and advanced material is collected in the **appendix** at the end of the deck.

Notation in this module

Symbol	Meaning
y_t	the observation at time t; order matters
y_{t-k}	the k-th *lag* of the series
r_k	sample autocorrelation at lag k (the ACF)
ϕ	AR coefficient(s)
AR(p)	regression of y_t on its own last p values
$\hat{y}_{T+h\|T}$	forecast of horizon h made with data up to T
ε_t	white noise: mean 0, no autocorrelation
MAE	mean absolute error of a forecast

Sources and references

Primary textbook

James, Witten, Hastie, Tibshirani & Taylor (2023), *An Introduction to Statistical Learning, with Applications in Python*, Springer. — `Bikeshare` and `Weekly` are its bundled datasets; ISLP §10.5 sketches sequences, this module develops them.

Canonical further reading (optional)

Hyndman & Athanasopoulos, *Forecasting: Principles and Practice* (3rd ed., online) — the standard applied reference; everything here scales up there.

Learning objectives

By the end of this module you will be able to:

- **Explain** why i.i.d. reasoning fails on ordered data, and what replaces it (stationarity, autocorrelation).
- **Compute** and read an ACF — by hand on six numbers, and in code on 8,645.
- **Fit** an AR model as a Chapter 3 regression on lagged copies, and state when it is stationary.
- **Beat** the two baselines every forecast must beat: the global mean and the seasonal naive.
- **Evaluate** honestly with a rolling-origin backtest — and say precisely what shuffled cross-validation gets wrong.

Roadmap of this module

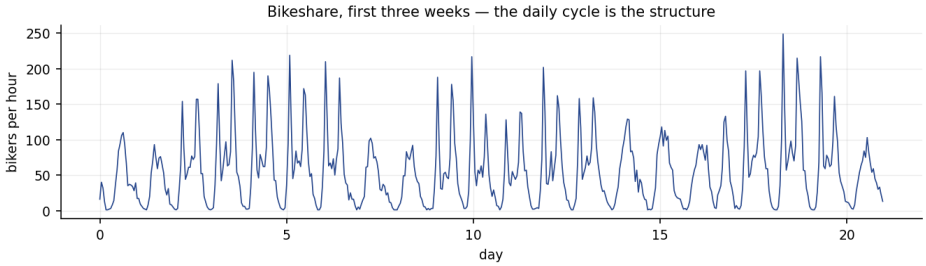
From plotted order to honest forecasts

1. The problem: rows that remember — and Chapter 5's unanswered question.
2. Seeing structure: the series, its seasonal profiles, the ACF.
3. AR models: least squares on the past, stationarity, baselines.
4. Honest evaluation: rolling-origin backtests vs shuffled CV.
5. A financial postscript: why returns resist all of this.
6. Python lab, summary, appendix.

Outline

1. The Problem
2. Seeing Structure
3. AR Models
4. Honest Evaluation
5. A Financial Postscript
6. Python Lab
7. Summary

One year of hourly bike rentals



The order is the signal

`Bikeshare`: 8,645 hourly counts, mean 144, max 651. Chapter 4 met this dataset as exchangeable rows; plotted *in order* it is almost periodic. Shuffle the rows and this structure — most of the predictability — is destroyed.

What independence buys you — and what its loss costs

	i.i.d. rows (Ch. 1–8)	**time series**
information per row	full	partly shared with neighbours
train/test split	any random split	*past* trains, *future* tests
CV (Ch. 5)	shuffle freely	shuffling leaks the future
the mean	the natural predictor	often the *worst* sane predictor
new data	more of the same	possibly a different regime

The one new assumption

In place of independence we assume **stationarity**: the joint distribution of (y_t, y_{t+k}) does not depend on t — levels, spreads and correlations are stable over time. It plays the role exchangeability played in module A3: the licence for learning from the past at all.

Exercise A11.1 — Where does the leak happen? [Concept]

Exercise

A colleague fits a model to three years of daily sales, evaluates it with shuffled 10-fold CV from Chapter 5, reports MAE = 41, deploys — and measures MAE = 70 in production.

1. Explain the mechanism of the failure in two sentences: what exactly did the shuffled folds let the model see?
2. Their model uses only *calendar features* (weekday, month) — no lags. Does the argument change?
3. Propose the evaluation they should have run.

Solution — Exercise A11.1

Solution

Part 1. In a shuffled fold, the days immediately before and after each test day sit in the training set. Sales on neighbouring days are strongly correlated, so the model effectively *interpolates between known neighbours* — a much easier task than the production task, which is *extrapolating beyond the last known day*. The CV number answers the wrong question.

Part 2. It softens but does not vanish. With purely calendar features the model cannot copy neighbours directly — but the folds still share the *level* of each local stretch (this year's demand regime), so drifting levels and new regimes are still invisible to shuffled CV. The gap of 41 $\rightarrow$ 70 typically has both components.

Part 3. A **rolling-origin backtest**: train on data up to time T, forecast the next horizon, roll T forward, repeat; report the average error over origins. It rehearses deployment exactly.

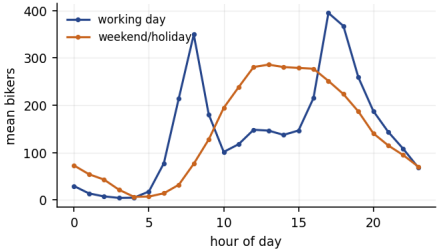
Common mistake

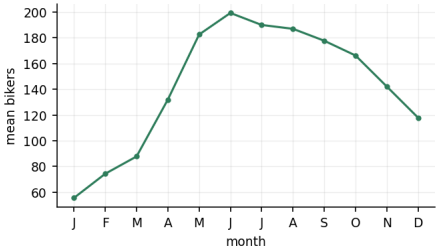
"We used cross-validation, so the estimate is honest." Chapter 5's guarantee rests on exchangeable rows; time-ordered rows are the textbook case where it breaks.

Outline

1 The Problem
2 Seeing Structure
3 AR Models
4 Honest Evaluation
5 A Financial Postscript
6 Python Lab
7 Summary

Two clocks: the day and the year





Seasonality is just a curve over a clock

Working days peak at **hour 17** with a morning twin at 8 — commuting. Weekends have one broad afternoon hump. Months carry a second, annual cycle. Each profile is a Chapter 7 curve fitted over a circular clock — nothing beyond the taught course.

The autocorrelation function: correlation with your own past

Rule

$$r_k = \frac{\sum_{t=k+1}^{T}(y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^{T}(y_t - \bar{y})^2} \qquad k = 1, 2, \ldots$$

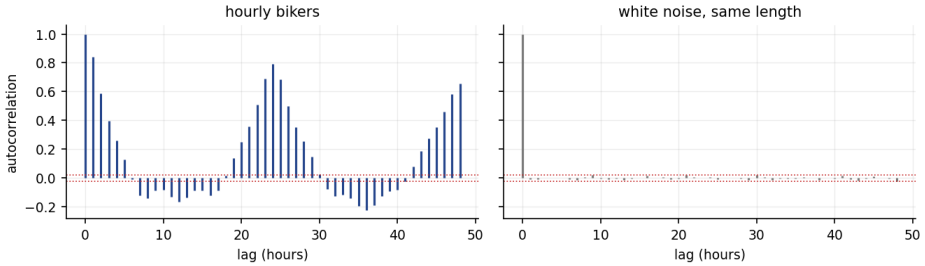
How to read this

It is Chapter 3's correlation coefficient, computed between the series and a k-shifted copy of itself (one shared normaliser). $r_k \approx 0$ at all lags $\Rightarrow$ white noise, nothing to forecast. For pure noise, $|r_k| < 1.96/\sqrt{T}$ about 95% of the time — the dotted band on every ACF plot (± 0.021 here).

Bikeshare's memory

$r_1 = 0.84$: this hour looks like last hour. $r_{24} = 0.79$: this hour looks like the same hour yesterday. Both dwarf the noise band — two separate memories, and an AR model should be offered *both*.

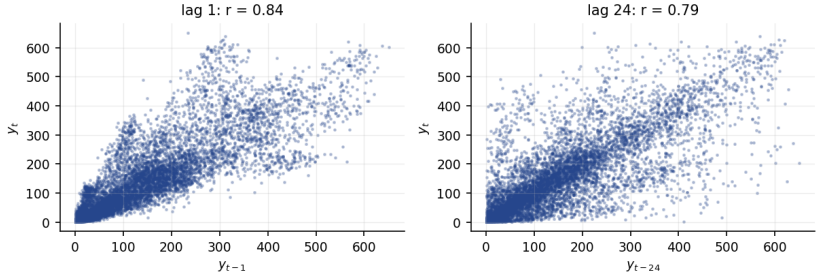
The ACF, drawn



Structure vs nothing

Left: bikers — slow decay from $r_1 = 0.84$ plus a clean spike ridge at lag 24 and its multiples. Right: white noise of the same length — everything inside ± 0.021. The ACF is the module's diagnostic instrument: it tells you *whether* and *where* the past is informative.

The same fact as a scatter plot



Lag plots are Chapter 2 pictures

Plot y_t against y_{t-1} ($r = 0.84$) or against y_{t-24} ($r = 0.79$) and forecasting collapses into the course's oldest picture: a supervised scatter with a strong linear signal — the predictors just happen to be the outcome's own past.

Exercise A11.2 — An ACF by hand [Math]

Exercise

A tiny series: $y = (3,\ 5,\ 4,\ 6,\ 5,\ 7)$.

1. Compute $\bar{y}$ and the deviations.
2. Compute r_1 and r_2 from the formula (shared denominator).
3. One of the two is negative and one clearly positive. Describe the pattern in the data that produces exactly this pair.

Solution — Exercise A11.2

Solution

Part 1. $\bar{y} = 30/6 = 5$; deviations $d = (-2, 0, -1, 1, 0, 2)$.

Part 2. Denominator $\sum d_t^2 = 4 + 0 + 1 + 1 + 0 + 4 = 10$.

$$r_1 = \frac{(-2)(0)+(0)(-1)+(-1)(1)+(1)(0)+(0)(2)}{10} = \frac{-1}{10} = -0.1,$$

$$r_2 = \frac{(-2)(-1)+(0)(1)+(-1)(0)+(1)(2)}{10} = \frac{4}{10} = +0.4.$$

Part 3. The series zig-zags: each value overshoots then undershoots a rising path — so adjacent values are (mildly) anti-correlated, while values two apart sit on the same side of the zig-zag and agree. A period-2 alternation shows up as exactly this sign pattern, the miniature of Bikeshare's period-24 ridge.

Common mistake

Recomputing the denominator on the shortened overlap for each lag. The standard ACF divides every lag by the *same* full-series sum of squares — that is what makes r_k comparable across k.

Outline

1 The Problem
2 Seeing Structure
3 AR Models
4 Honest Evaluation
5 A Financial Postscript
6 Python Lab
7 Summary

The autoregression: Chapter 3 pointed at the past

Rule

$$\text{AR}(p): \quad \boxed{y_t = \beta_0 + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \varepsilon_t}$$

Build the lagged copies as columns, then fit by **ordinary least squares**.

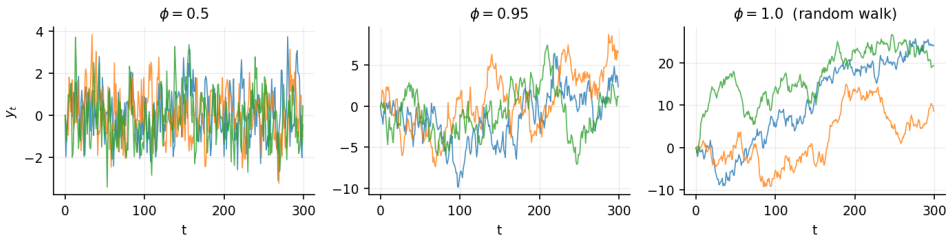
How to read this

There is no new estimator here: make a design matrix whose columns are $y_{t-1}, y_{t-2}, \ldots$ and run Chapter 3. The art moves into *which lags to offer*: the ACF said Bikeshare has an hourly memory **and** a daily one, so we offer lags $1, 2$ (recent hours), $24, 25$ (yesterday) and 168 (last week).

The fitted coefficients

On lags $(1, 2, 24, 25, 168)$: $\hat{\phi} = (0.83, -0.20, 0.51, -0.38, 0.18)$ — last hour dominates, yesterday's same hour adds half a weight, last week a little more.

Stationarity: when does an AR(1) settle down?



The knife edge at $\phi = 1$

For $|\phi| < 1$ the series forgets: shocks decay like ϕ^k, the mean $\beta_0/(1-\phi)$ exists, forecasts converge to it. At $\phi = 1$ — the **random walk** — shocks never decay, the variance grows without bound, and the best forecast of every horizon is *today's value*. Financial prices live at this edge; that is the postscript.

Exercise A11.3 — AR(1) arithmetic [Math]

Exercise

A stationary AR(1): $y_t = 2 + 0.8\, y_{t-1} + \varepsilon_t$, and today $y_T = 14$.

1. Compute the long-run mean of the process.
2. Compute the one- and two-step forecasts $\hat{y}_{T+1|T}$ and $\hat{y}_{T+2|T}$.
3. Where do the forecasts head as the horizon grows, and why is that the honest answer?

Solution — Exercise A11.3

Solution

Part 1. Take expectations of both sides at stationarity, $\mu = 2 + 0.8\mu$, so $\mu = 2/(1 - 0.8) = 10$.

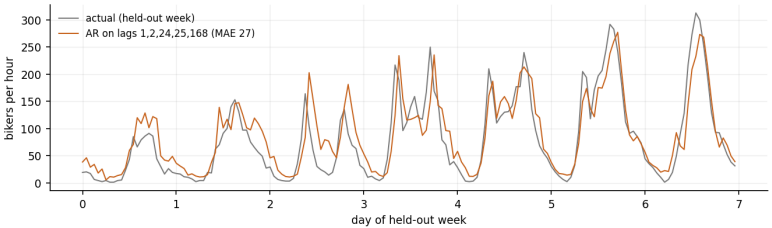
Part 2. Set future noise to its mean (zero): $\hat{y}_{T+1|T} = 2 + 0.8 \times 14 = 13.2$; $\hat{y}_{T+2|T} = 2 + 0.8 \times 13.2 = 12.56$.

Part 3. Each step multiplies the distance from μ by 0.8: $14 \to 13.2 \to 12.56 \to \cdots \to 10$. The forecast decays geometrically to the mean because the model's memory does — beyond its memory horizon the honest forecast *is* the mean, with widening uncertainty. A forecast that stays interesting at every horizon is advertising, not statistics.

Common mistake

Forecasting y_{T+2} by plugging in the *observed* y_{T+1} — it does not exist yet. Multi-step forecasts feed the model its *own* predictions, which is exactly why errors compound with horizon.

Forecasting the held-out week



Beat the baselines or go home

Final week held out, model fitted strictly on the past. MAE per hour: global mean 90.5, seasonal naive (same hour last week) 32.7, our AR on lags $(1, 2, 24, 25, 168)$: 27.0.

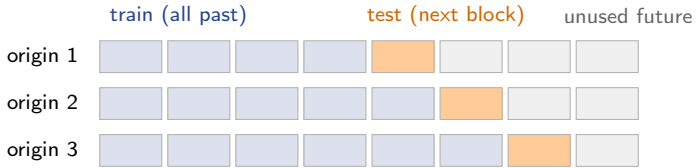
The seasonal naive is the baseline that matters

Copying last week already crushes the mean. Any model you propose must beat *that* — many published forecasts do not.

Outline

1. The Problem
2. Seeing Structure
3. AR Models
4. Honest Evaluation
5. A Financial Postscript
6. Python Lab
7. Summary

Rolling-origin backtesting — Chapter 5, made time-safe



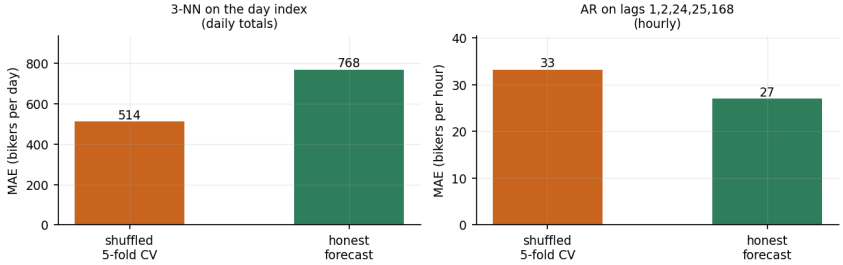
Algorithm (rolling origin)

For each origin $T_1 < T_2 < \ldots$: fit on data up to T_i, forecast the next block, record the error; report the average. Every fold trains on the past and tests on the future — deployment, rehearsed.

This answers Chapter 5's open question

Chapter 5's pitfalls slide asked how to cross-validate a time series. This is the answer: the folds *roll forward*; they are never shuffled.

The leak, measured



Shuffled CV answers a different question

Left, a 3-NN on the day index (daily totals): shuffled CV says 514; honestly forecasting the final 60 days costs 768 — the CV number flatters by a third, because shuffled folds let the model interpolate between neighbouring days. Right, the AR on lags: 33.1 vs 27.0 — with honest lag features the two nearly agree, and CV even errs *pessimistic*. The gap is not a fixed tax; it depends on what the model can reach — so you cannot correct for it, only avoid it.

Exercise A11.4 — The seasonal naive as a null model [Concept]

Exercise

Your team's gradient-boosted demand model reports backtested MAE $= 31$; the deck omits baselines.

1. Define the two baselines you would demand, and what each one costs to run.
2. On Bikeshare's held-out week those baselines score 90.5 and 32.7. Rewrite the team's headline honestly given their 31.
3. Name the Chapter 2 concept that explains why the naive baselines are so strong here.

Solution — Exercise A11.4

Solution

Part 1. The *global mean* (one number; free) and the *seasonal naive* $\hat{y}_t = y_{t-s}$ (copy the last cycle; also free). Both need zero fitting and one line of code.

Part 2. "Our model improves on copying last week by 1.7 MAE points (32.7 $\rightarrow$ 31, about 5%) — and on the unconditional mean by 66%." The second number sounds impressive and is nearly all seasonality; the first is the model's actual contribution.

Part 3. The bias–variance decomposition's *irreducible-error* lesson, in time-series clothing: most of the predictable variance lives in the seasonal cycle, which the naive already captures at zero variance cost. The model competes only for the remainder.

Common mistake

Reporting improvement over the *mean* when a seasonal naive exists. On strongly seasonal data that comparison flatters any model, including broken ones.

Extended Exercise A11.1 — Build the forecaster [Python]

Extended exercise (15 min)

Reproduce the module's forecaster from scratch on `Bikeshare.csv`:

1. Build lagged columns $(1, 2, 24, 25, 168)$ of `bikers`; hold out the final 168 hours; fit OLS on the rest.
2. Compute held-out MAE for your model, the seasonal naive (y_{t-168}) and the global mean. Target: $\approx$ 27 / 32.7 / 90.5.
3. Add the hour-of-day as 23 dummy variables (Chapter 3 machinery) to the lag design. Does it help on top of the lags? Why is the answer close to no?
4. Refit dropping lag 168. Which baseline does your model now barely beat, and what does that teach about feature choice?

Solution — Extended Exercise A11.1 (1/2)

Solution — code

```
import numpy as np, pandas as pd
bike = pd.read_csv("Bikeshare.csv", index_col=0)
y = bike["bikers"].to_numpy(float)
lags = [1, 2, 24, 25, 168]; p = max(lags)
X = np.column_stack([y[p-k : len(y)-k] for k in lags])
yy = y[p:]
cut = len(yy) - 168 # final week held out
Xtr = np.column_stack([np.ones(cut), X[:cut]])
beta = np.linalg.lstsq(Xtr, yy[:cut], rcond=None)[0]
pred = np.column_stack([np.ones(168), X[cut:]]) @ beta
mae = np.abs(yy[cut:] - pred).mean() # ∼27.0
naive = np.abs(yy[cut:] - yy[cut-168:cut]).mean() # ∼32.7
mean_ = np.abs(yy[cut:] - yy[:cut].mean()).mean() # ∼90.5
print(f"AR {mae:.1f} | naive {naive:.1f} | mean {mean_:.1f}")
```

Solution — Extended Exercise A11.1 (2/2)

Solution — parts 3 and 4

Part 3. The hour dummies add almost nothing: lags 24, 25 and 168 already *carry* the daily shape, measured on the most recent cycles rather than averaged over the year. Calendar features and seasonal lags are two encodings of the same clock — offering both mostly buys collinearity (Chapter 3's warning), not accuracy.

Part 4. Without lag 168 the model loses the weekly rhythm — Saturdays are forecast from Friday's shape. The MAE degrades toward the seasonal naive's 32.7: the model now barely beats *copying*, though it still crushes the mean. Feature choice in AR models **is** the modelling; the estimator never changed.

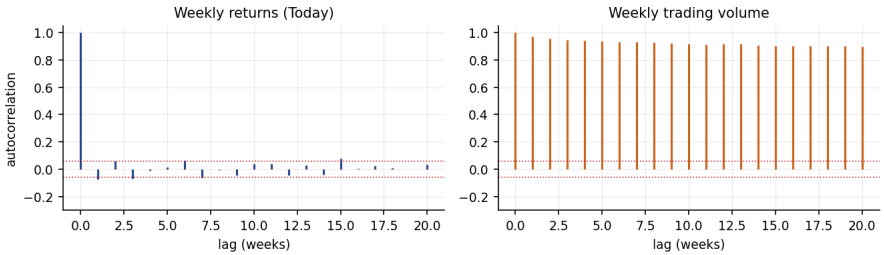
Common mistake

Standardising or shuffling the design matrix out of habit before the split. The split must come first and must be temporal — every preprocessing statistic belongs to the training window (Chapter 5's leakage rule, sharpened by time).

Outline

1 The Problem
2 Seeing Structure
3 AR Models
4 Honest Evaluation
5 A Financial Postscript
6 Python Lab
7 Summary

Why Chapter 4 could barely predict the market



Two series from the same file, two different worlds

`Weekly` returns: $r_1 = -0.075$, inside the ± 0.059 band almost everywhere — close to white noise, which is *why* Chapter 4's classifiers hovered at chance. Trading *volume*: $r_1 = 0.973$, extremely forecastable. Efficient markets erase autocorrelation in **returns** (any pattern would be traded away), but not in activity.

Exercise A11.5 — Reading the two ACFs [Concept]

Exercise

Using the previous slide's plots:

1. A consultant proposes an AR(5) on `Weekly` returns to time the market. Expected out-of-sample R^2, roughly — and why?
2. The same consultant proposes an AR model for next week's trading *volume*. Reasonable?
3. Connect this to the random-walk panel of the stationarity slide: which object in finance is (approximately) the random walk — prices, returns, or volume?

Solution — Exercise A11.5

Solution

Part 1. Roughly zero. The lag-1 autocorrelation of -0.075 caps the linear predictability: an AR(1) explains about $r_1^2 \approx 0.6\%$ of the variance in-sample, and even that mostly evaporates out of sample. This is the same wall `Smarket` hit in Chapter 4 — it is a property of the data, not of the model.

Part 2. Yes — $r_1 = 0.973$ means volume is highly persistent; a small AR fits it well. Not everything financial is unforecastable: only the part whose forecastability someone could monetise.

Part 3. *Prices*: if returns (the differences) are white noise, the price level is their running sum — a random walk, the $\phi = 1$ panel. Differencing a random walk yields the stationary object; that is why one models returns, never price levels.

Common mistake

Fitting on price *levels* and celebrating $R^2 = 0.99$. A random walk regressed on its own lag always yields $R^2 \approx 1$ and forecasts nothing; the information is in the differences.

Extended Exercise A11.2 — Design the backtest [Integrative]

Extended exercise (15 min)

A retailer wants daily demand forecasts, horizon 14 days, for 200 stores; promotions are known in advance; the assortment changed structurally 10 months ago. Design the evaluation:

1. Specify the backtest: window type (expanding or sliding), origin spacing, horizon, and metric — with one sentence of justification each.
2. Which two baselines go in every report?
3. The structural break: how does it change your training window, and what does it do to the stationarity assumption?
4. A colleague proposes tuning hyperparameters with shuffled CV *inside* each training window, keeping the outer backtest temporal. Legitimate or leak?

Solution — Extended Exercise A11.2

Solution

Part 1. *Sliding* window (the break makes ancient data misleading); origins every 7 days (dense enough to average over weekday effects, cheap enough to run); horizon exactly 14 days ahead (evaluate what you will deploy); MAE per store-day, aggregated per horizon step (day-1 and day-14 accuracy differ and both matter).

Part 2. Global (or per-store) mean, and the seasonal naive y_{t-7}. Improvements are quoted against the naive.

Part 3. Either start training after the break or down-weight the old regime; stationarity holds at best *within* regimes. Ten months of daily data per store is workable. The backtest must also not average across the break, or it scores a regime nobody will face again.

Part 4. Legitimate as a pragmatic approximation — the future relative to each *outer* test block is untouched, so the reported number stays honest. But the tuned hyperparameters may still favour interpolators (part 3 of the leak story), so temporal inner folds are the safer default.

Outline

1. The Problem
2. Seeing Structure
3. AR Models
4. Honest Evaluation
5. A Financial Postscript
6. Python Lab
7. Summary

Python lab — run it live

Reproduce every number: `make_figures.py`

Every number on these slides is computed by `make_figures.py` in this module's folder, from the bundled `Bikeshare.csv` and `Weekly.csv` — run it and read it alongside the deck:

- the from-scratch ACF ($r_1 = 0.84$, $r_{24} = 0.79$, band ± 0.021);
- the AR-on-lags forecaster and its baselines (27.0 / 32.7 / 90.5);
- the leak experiment: shuffled CV 514 vs honest 768 on daily totals;
- the `Weekly` postscript ($r_1 = -0.075$ returns, 0.973 volume).

A companion notebook in the course lab format is in preparation; until it lands, the script *is* the lab — it runs top to bottom in the course environment with no extra packages.

The whole module in fourteen lines

```
import numpy as np, pandas as pd
bike = pd.read_csv("Bikeshare.csv", index_col=0)
y = bike["bikers"].to_numpy(float)

def acf(x, k): # correlation with your own past
    d = x - x.mean()
    return (d[k:] * d[:-k]).sum() / (d * d).sum()
print(f"r1 = {acf(y,1):.2f}, r24 = {acf(y,24):.2f}") # 0.84, 0.79

lags = [1, 2, 24, 25, 168]; p = max(lags) # AR = Ch.3 on lags
X = np.column_stack([np.ones(len(y)-p)]
                    + [y[p-k:len(y)-k] for k in lags])
beta = np.linalg.lstsq(X[:-168], y[p:-168], rcond=None)[0]
mae = np.abs(y[-168:] - X[-168:] @ beta).mean()
print(f"held-out week MAE = {mae:.1f}") # 27.0
```

Outline

1. The Problem
2. Seeing Structure
3. AR Models
4. Honest Evaluation
5. A Financial Postscript
6. Python Lab
7. Summary

Module A11 in one slide

What we accomplished

Took the course's machinery to ordered data — and replaced the one habit (shuffling) that ordered data forbid.

- The order is the information: Bikeshare's daily and weekly clocks dominate everything.
- The ACF measures memory: $r_1 = 0.84$, $r_{24} = 0.79$ against a ± 0.021 noise band — computed by hand and in code.
- AR(p) is least squares on lagged copies; stationarity ($|\phi| < 1$) is what makes it meaningful.
- Baselines first: mean 90.5, seasonal naive 32.7, our AR 27.0 MAE on a truly held-out week.
- Evaluation must roll forward: shuffled CV said 514 where reality said 768 on daily totals.
- Financial returns are the null case: $r_1 = -0.075$ — nothing to forecast, by design of the market.

Key formulas at a glance

ACF $r_k = \sum_{t>k}(y_t - \bar{y})(y_{t-k} - \bar{y}) / \sum_t (y_t - \bar{y})^2$; noise band $\pm 1.96/\sqrt{T}$.

AR(p) $y_t = \beta_0 + \sum_j \phi_j \, y_{t-k_j} + \varepsilon_t$, fitted by OLS on lagged columns.

AR(1) mean $\mu = \beta_0/(1-\phi)$, provided $|\phi| < 1$.

AR(1) forecast $\hat{y}_{T+h|T} = \mu + \phi^h (y_T - \mu)$ — geometric decay to the mean.

Seasonal naive $\hat{y}_t = y_{t-s}$ (here $s = 168$ hours).

Random walk $\phi = 1$: forecast = today, variance grows with h; difference it to get stationarity.

Vocabulary checklist

Term	Meaning
lag	the series shifted back k steps
autocorrelation (ACF)	correlation between y_t and y_{t-k}
white noise	no autocorrelation at any lag; unforecastable
stationarity	distribution of (y_t, y_{t+k}) free of t
AR(p)	regression of y_t on its last p values
random walk	AR(1) with $\phi = 1$; difference before modelling
seasonal naive	forecast by copying the last full cycle
horizon	how far ahead a forecast reaches
rolling-origin backtest	train on past, test on next block, roll forward
leakage (temporal)	any future information touching the training side

Decision rules of thumb

- Plot the series in order before anything else; then plot the ACF.
- ACF flat inside $\pm 1.96/\sqrt{T} \Rightarrow$ stop — report the mean and its uncertainty; there is nothing to model.
- Offer the model the lags the ACF points at (recent, one cycle back, one long cycle back) — not every lag up to 200.
- Trend or unit root in sight $\Rightarrow$ difference first (returns, not prices).
- Always run the two free baselines; quote your gain against the *seasonal naive*.
- Evaluate with a rolling origin at the deployment horizon. Never shuffle.
- Preprocessing statistics (means, scalers) come from the training window only.

Common pitfalls

Don't

1. Shuffle time-ordered rows into CV folds and call the score honest.
2. Report improvement over the global mean when a seasonal naive exists.
3. Fit levels of a random walk and admire the R^2.
4. Feed multi-step forecasts observed future values instead of their own predictions.
5. Fit one model across a structural break as if the regime never changed.
6. Read a long-horizon forecast that never widens as anything but overconfidence.

Self-check questions

Quick self-assessment

1. For $y = (3, 5, 4, 6, 5, 7)$: r_1 and r_2?
2. An AR(1) has $\beta_0 = 2$, $\phi = 0.8$, $y_T = 14$. Long-run mean and the next two forecasts?
3. Bikeshare's held-out-week MAEs were 90.5 / 32.7 / 27.0. Which estimator is which, and which comparison should a report headline?
4. Why did shuffled CV report 514 where the honest forecast cost 768 — and why did the same comparison for the AR model nearly agree?
5. `Weekly` returns have $r_1 = -0.075$ with band ± 0.059. What follows for market-timing models?
6. What single transformation turns a random walk into something stationary?

Answers

(1) -0.1 and $+0.4$. (2) $\mu = 10$; 13.2, then 12.56. (3) Global mean / seasonal naive / AR on lags; headline the gain over the *naive* ($32.7 \to 27.0$). (4) Shuffled folds let the 3-NN interpolate between neighbouring days; the AR's lag features grant no such shortcut, so its two scores nearly coincide — the leak's size depends on the model. (5) Linear predictability is capped near $r_1^2 \approx 0.6\%$ — practically nil. (6) Differencing: model the changes, not the levels.

Five things to remember

Module A11 in five lines

1. **The order is the information** — shuffling destroys the very structure you would model.
2. **The ACF is the instrument:** 0.84 at lag 1 and 0.79 at lag 24 told us which lags to offer before any model was fitted.
3. **An AR model is Chapter 3 on lagged copies** — the estimator is old; the feature choice is the craft.
4. **Beat the seasonal naive or say so** — $32.7 \rightarrow 27.0$ is the honest headline; $90.5 \rightarrow 27.0$ is mostly the clock.
5. **Evaluate by rolling forward, never by shuffling** — the leak (514 vs 768) is unfixable after the fact.

What's next

- **Module A3 — Conformal Prediction** named time order as the thing that breaks exchangeability; this module is what you do about it. Conformal variants for time series exist — built on the rolling origins you now run.
- **Module A10 — Causal Inference from Observational Data**: difference-in-differences is a time-series design at heart; parallel trends is a statement about two series' common structure.
- **Chapter 8's boosting** plus this module's lag features is the workhorse forecaster in industry — the design matrix is the same one you built here.

Appendix — what is in here, and why

Nothing in the appendix is needed to follow the module: each slide is a formal derivation, a longer worked exercise, or a side topic. Read it when you want the full story, or when a later chapter sends you back here.

Contents of the appendix

- **AR(1) mean and variance, derived** — two lines each; moved out because the results, not the algebra, carry the main deck.
- **From AR to ARMA and friends** — a map of the classical model zoo and what each letter buys; look-up material, not narrative.
- **Forecast intervals** — why uncertainty must widen with the horizon, and the AR(1) formula that widens it.

AR(1) moments, derived

Stationary AR(1): $y_t = \beta_0 + \phi y_{t-1} + \varepsilon_t$, $\mathrm{Var}(\varepsilon_t) = \sigma^2$, $|\phi| < 1$.

Mean

Expectations of both sides: $\mu = \beta_0 + \phi\mu \Rightarrow \mu = \beta_0/(1-\phi)$.

Variance and ACF

Variances of both sides (independence of ε_t from the past): $v = \phi^2 v + \sigma^2 \Rightarrow v = \sigma^2/(1-\phi^2)$. Multiplying the defining equation by y_{t-k} and taking expectations gives $\rho_k = \phi^k$ — geometric decay, the signature shape.

Why $\phi = 1$ destroys everything

Both denominators hit zero: no stationary mean, no stationary variance. Every formula on the main deck quietly assumed $|\phi| < 1$; this is where.

The classical zoo, in one table

Model	Adds	Use when
MA(q)	lagged *errors* as predictors	short, sharp memory
ARMA(p, q)	both memories	parsimony over a long AR
ARIMA(p, d, q)	d rounds of differencing	trends / unit roots
SARIMA	seasonal AR/MA terms at lag s	one dominant seasonal cycle
ETS	explicit trend + seasonal components	business series, automatic use

How to place this module in the zoo

Our lag-selected AR with seasonal lags (24, 168) is a hand-built cousin of SARIMA — same idea, plain OLS. `statsmodels.tsa` fits the full zoo; the diagnostics (ACF, baselines, rolling origin) transfer unchanged, and they are the part that decides whether *any* member of the zoo is working.

Forecast intervals widen — by law, not by taste

For a stationary AR(1), the h-step forecast error is $\sum_{j=0}^{h-1} \phi^j \varepsilon_{T+h-j}$, so

$$\operatorname{Var}\left(y_{T+h} - \hat{y}_{T+h|T}\right) = \sigma^2 \sum_{j=0}^{h-1} \phi^{2j} = \sigma^2 \, \frac{1-\phi^{2h}}{1-\phi^2} \xrightarrow[h\to\infty]{} \frac{\sigma^2}{1-\phi^2} = \operatorname{Var}(y).$$

The honest endpoint

As the horizon outruns the memory, the forecast variance climbs to the series' *own* variance: at long horizons you know nothing beyond the unconditional distribution — which is exactly what the point forecast (decay to μ) was already admitting. Module A3's conformal machinery can calibrate these intervals empirically from rolling-origin errors.